Last modified on

Sports Data Soccer API - In-Venue Aggregated Phases Feed (MA56)

Get in-venue aggregated match statistics across both teams and players.

Overview

The Soccer In-Venue Aggregated Phases feed provides aggregated match statistics across both teams and players.

> **Required Compliance:**
> To comply with our API security and best practice logic for your API calls please refer to our FAQs. We also recommend that you take a look at our API Overview and Authentication Guide. You will learn the basics of how our API works and key concepts to get started.

How Often Should I Poll The Feed?

We have provided some guidelines to help ensure that you call the feed only as often as you need to. Make sure that you don't exceed the rate confirmed with us - this can vary depending on whether you make requests as a B2B or B2C outlet - if you need more information on this, contact your Stats Perform representative.

POLLING FREQUENCY
TIER 13
Post Match

In-Venue Aggregated Phases (MA56)

USAGE	FEED
GET **invenue aggregated phases** by fixture UUID	`/soccerdata/invenueaggregatedphases/{outletAuthKey}/{fixtureUuid}?{queryPara`

> If your request URL is as follows (does not contain a fixture UUID), the feed returns an error: `/soccerdata/invenueaggregatedphases/{outletAuthKey}`

Query Parameters And Example Request Parameters

This feed also makes use of the Global query parameters, and you can combine them with the following query parameters (some of which can be used in conjunction with each other. However, you must query the `/invenueaggregatedphases` feed by a specific fixture (match) UUID.

How to use query parameters and make successful requests:
In our guides, we use HTTPS syntax for our example `GET` requests (rather than cURL, for example). All URLs are case-sensitive - this means that URL strings must contain the correct case and operators. You MUST format your requests correctly and include certain information; otherwise, an error will be returned. You can find information about the API, including the base URL (`{protocol}://{requestDomain}/{feedResource}`), in the main guide and API Overview and Authorisation guide.

- Include the following information in the URL or header and make sure it is correct for your outlet: your unique and valid `{outletAuthKey}` this must be after the Outlet Authentication Key (and endpoint, if applicable). for `_rt={mode}` (operating mode) and `_fmt={dataFormat}` (response format), entity query parameter and UUID.
- Begin the query parameter string with `?` (question mark) - this must be after the Outlet Authentication Key (and endpoint, if applicable). To build up a query string, make sure that each query parameter is separated by the `&` (ampersand) operator. Values must be separated by the correct operator, for example `&` (multiple values, 'AND'), or - if a parameter supports it - `,` (multiple values, 'OR') or `!` ('NOT').
- Use the required `_` (underscore) for any parameters listed with this prefix.
- Remove any placeholder value curly braces `{}` from your calls (we include these as an example only).
- If using the JSONP format (`_fmt=jsonp`) you MUST use it in combination with the callback parameter (`_clbk`) and a fixed value (or where you use a different value in a different instance, you must do this as little as possible). Be aware that common JavaScript frameworks, such as jQuery, can default to use randomly-generated function names - you must make sure that these are overridden and define a fixed function name instead.

Query parameter	Value and description
`{fixtureUuid}` Fixture UUID	GET **invenue aggregated phases** by fixture UUID (path parameter method) `GET https://api.performfeeds.com/soccerdata/invenueaggregatedphases/{outletAuthKey}/{fixtureUuid}?_fmt={dataFormat}&_rt={mode}`
`fx`	GET **invenue aggregated phases** by fixture UUID (fx query parameter method). `GET https://api.performfeeds.com/soccerdata/invenueaggregatedphases/{outletAuthKey}?fx={fixtureUuid}&_rt={mode}&_fmt={dataFormat}`
`{fixtureUuid} + _lcl`	GET **invenue aggregated phases** by fixture UUID, language code. `GET https://api.performfeeds.com/soccerdata/invenueaggregatedphases/{outletAuthKey}/{fixtureUuid}?_lcl={languageCode}&_fmt={dataFormat}&_rt={mode}`

If `_lcl` parameter is not set `en-us` language code is used. No asterisk * allowed an in other feeds.

The `eventContentType` is coma separated set of types that are returned. One or more of `adMarker`, `insight`, `insightBetting`, `genericGraphics`, `autoLtc`, `nlgPreview`, `customGraphics`, `manualLtc`

Sample Calls

These sample calls let you see an example URL and the response - click on the links below (hyperlinks open in a new window).

- XML
- JSON

Feed Output

Here, you can find an explanation of all possible response fields and data. This is XML but fields in the feed response are similar for JSON.

▸<inVenueAggregatedPhases> (XML only)

▸<matchInfo>

▸<description>

▸<sport>

▸<ruleset>

▸<competition>

▸<country>

▸<tournamentCalendar>

▸<stage>

▸<contestants> (XML only)

▸<contestant>

▸<country>

▸<venue>

▸<liveData>

▸<matchDetails>

▸<periods> (XML only)

▸<period>

▸<scores>

▸<ht>

▸<ft>

▸<total>

▸<goal>

▸<lineUp>

▸<player>

▸<touchAggregates>

▸<stat>

▸<firstTouchAggregates>

▸<stat>

▸<lineBreakingActionAggregates>

▸<stat>

▸<overloadAggregates>

▸<stat>

▸<passingAggregates>

▸<stat>

▸<markingAggregates>

▸<stat>

▸<matchOfficial>

▸<teamStats>

▸<phaseAggregates>

▸<stat>

▸<outOfPossessionCompactnessAggregates>

▸<stat>

▸<inPossessionCompactnessAggregates>

▸<stat>

Phase Definitions

Below are brief explanations of, entries in the MA56 feed. This feed aggregates information across the entire game, rather than aggregations within individual Phases.

- `teamStats` : aggregations at a team level over the full match
  - `phaseAggregates` : aggregations specifically regarding the Phase labels as defined here.
    - `number{PHASE NAME}` (int): number of times that the the phase `PHASE NAME` occurs during the match.
    - `number{PHASE NAME}LeadingToGoal` (int): number of times that the the phase `PHASE NAME` occurs during the match where the phase includes a goal.
    - `number{PHASE NAME}LeadingToShot` (int): number of times that the the phase `PHASE NAME` occurs during the match where the phase includes a shot.
    - `timeIn{PHASE NAME}` (float): total time in milliseconds that spent in the phase `PHASE NAME` .
    - `percentageTimeIn{PHASE NAME}` (float): percentage (0–100) of time that the team was in PHASE NAME whilst they had possession of the ball.
  - `outOfPossessionCompactnessAggregates` (int): aggregates for the team over the match whilst they were out of possession (the defending team).
    - `timeHighDefensiveCompactness` , `timeMediumDefensiveCompactness` , `timeLowDefensiveCompactness` (int): time in milliseconds spent in each of the defensive compactness labels as defined under Defensive Compactness here.
    - `averageDefensiveAreaCoverage` (float): average Convex Hull area covered covered by the defensive team during the phase (in meters squared).
    - `averageTeamHorizontalWidth` (float): average width of the team in meters when out of possession (maximum y-coordinate difference between the outfield players for each frame).
    - `averageTeamVerticalLength` (float): average length of the team in meters when out of possession (maximum x-coordinate difference between the outfield players for each frame).
    - `averageHeightLastDefender` (float): average minimum y-coordinate of outfield players in the team whilst out of possession (defending).
  - `inPossessionCompactnessAggregates` : aggregates for the team over the match whilst they were in possession (the attacking team).

    - `averageTeamHorizontalWidth` (float): average width of the team in meters when in possession (maximum y-coordinate difference between the outfield players for each frame).
    - `averageTeamVerticalLength` (float): average length of the team in meters when in possession (maximum x-coordinate difference between the outfield players for each frame).
    - `averageHeightLastDefender` (float): average minimum y-coordinate of outfield players in the team whilst in possession (attacking).
  - `overloadAggregates`
    - `total` (int): the total number of overloads the team had whilst in-possession where overloads are defined under Overload here.
  - `lineBreakingActionAggregates`
    - `total` (int): total number of line-breaking passes and line-breaking carries.
    - `defenceLineBroken` (int): total number of line-breaking passes and line-breaking carries which break the final line (defensive line) of the defending team.
  - `passingAggregates`
    - `numberDirectPassing` (int): number of instances of direct passing play as defined under Direct Passing Play here.
    - `numberFastPassingTempo` (int): number of instances of fast passing tempo as defined under Fast Passing Tempo here.
  - `markingOppositionAggregates`
    - `percentagePlayersMarkingOpposition` (float): percentage (0–100) of the team's out of possession player-frame combinations in which a defender is marking an opponent.
      - Defined as `number of player marking frames` / `(number of out of possession frames * 10)` e.g. 5 outfield players are marked on average in 50% of all 30,000 out of possession frames = (5 * 15000) / (30,000 * 10) = 25%.
    - `averageIntensity` (float): When marking their opponents, what is the average marking intensity (a metric on a 0-1 scale, where 1 is highest intensity and 0 is lowest)
    - `averageDistance` (float): When marking their opponents, what is the average marking distance in meters.
  - `beingMarkedAggregates`
    - `percentagePlayersBeingMarked` (int): percentage (0–100) of the team's in possession player-frame combinations which are being marked by their opponents.
      - Defined as `number of player marking frames` / `(number of in of possession frames * 10)` e.g. 5 outfield players are marked on average in 50% of all 30,000 out of possession frames = (5 * 15000) / (30,000 * 10) = 25%.
    - `averageIntensity` (float): When being marked by their opponents, what is the average marking intensity (a metric on a 0-1 scale, where 1 is highest intensity and 0 is lowest).
    - `averageDistance` (float): When being marked by their opponents, what is the average marking distance in meters.
- `players`
  - `playerStats`: aggregations at a player level over the full match
    - `touchAggregates`
      - `{PHASE NAME}` (int): Total number of Opta Touch Events that this player has whilst their team is in each of the Phase labels as defined here.
    - `firstTouchAggregates`
      - `{PHASE NAME}` (int): Total number of Opta Touch Events that this player has whilst their team is in each of the Phase labels as defined here, which were the first Opta Touch Event during that phase by anyone on the possession team.
    - `initiatorAggregates`
      - `{PHASE NAME}` (int): (int): Total number of times that this player was the initiator for each of the Phase labels as defined here, where the initiator is defined here.
    - `overloadAggregates`
      - `totalInvolvements` (int): Total number of overloads during the match that occurred whilst this player's team was in possession and involved this player as an attacker. Overloads are defined under Overload here.
      - `totalOverloadsFaced` (int): Total number of overloads during the match that occurred whilst this player's team was out of possession and involved this player as a defender. Overloads are defined under Overload

here.

- `lineBreakingActionAggregates`
  - `total` (int): total number of Line Breaking Passes and Line Breaking Carries by this player
  - `defenceLineBroken` (int): total number of Line Breaking Passes and Line Breaking Carries by this player which break the final line (defensive line) of the defending team
- `passingAggregates`
  - `numberDirectPassing` (int): number of unique instances of direct passing play as defined under Direct Passing Play here, which this player was involved in.
  - `defenceLineBroken` (int): number of unique instances of fast passing tempo as defined under Fast Passing Tempo here , which this player was involved in.
- `markingOppositionAggregates`
  - `percentageTimeMarkingOpposition` (float): percentage (0–100) of the player's out of possession frames in which they are marking opponents.
    - Defined as `number of player marking frames` / `(number of player's out of possession frames * 10)` e.g. if the player has 5,000 out of possession frames and is marking an opposition player in 2,500 of them = 2,500/5,000 = 50%.
  - `averageIntensity` (float): When marking their opponents, what is the average marking intensity (a metric on a 0-1 scale, where 1 is highest intensity and 0 is lowest)
  - `averageDistance` (float): When marking their opponents, what is the average marking distance in meters
- `beingMarkedAggregates`
  - `percentageTimeBeingMarked` (float): percentage (0–100) of the player's in possession frames in which they are being marked by an opponents.
    - Defined as `number of player marking frames` / `(number of player's in possession frames * 10)` e.g. if the player has 5,000 in possession frames and is being marked by an opposition player in 2,500 of them = 2,500/5,000 = 50%.
  - `averageIntensity` (float): When being marked by their opponents, what is the average marking intensity (a metric on a 0-1 scale, where 1 is highest intensity and 0 is lowest)
  - `averageDistance` (float): When being marked by their opponents, what is the average marking distance in meters.